

progra ~~mmation~~ maths

Rapprocher les deux

<https://gwen.works/programath>

= VS ==

En maths, on note indifféremment le $=$ de la **déclaration** et le $=$ de l'**hypothèse**.

- Ce que j'appelle le $=$ de la *déclaration*, c'est celui qu'on utilise pour poser une variable : "Soit $a = 45$ ".
- Ce que j'appelle le $=$ de l'*hypothèse*, c'est le $=$ dont on n'est pas sûr, celui que l'on veut prouver ou réfuter : "Supposons $b = c + d$ ".

En programmation, ce que l'on dit est interprété par une machine, qui ne peut pas déduire cette différence cruciale toute seule.

On est donc obligé de noter les "deux $=$ " différemment :

Python	Math
<code>a = 45</code>	Soit $a = 45$
<code>a == 45</code>	$a = 45$
<code>P = a == 45</code>	Soit P la propriété " $a = 45$ "
<code>def P(n): return n == 2*n</code>	Soit $P(n) = (n = 2n)$

Comme en programmation on pose (ie on déclare) plus que l'on ne teste, le " $=$ " tout simple sert de " $=$ de la déclaration".

D'ailleurs, les propriétés c'est comme les relations, c'est des fonctions à valeurs dans $\mathbb{B}$ mais shhhh...

Syntaxe de base

$+$, $\times$ et tout ça.

<code>(1 + 2) * 3 - 4**6</code>	$(1 + 2) * 3 - 4^6$
<code>a//b + a/b</code>	$\lfloor \frac{a}{b} \rfloor + \frac{a}{b}$
<code>r = a%b</code>	Posons r tel que $a = \lfloor \frac{a}{b} \rfloor b + r$
<code>from math import sqrt sqrt(5)</code>	$\sqrt{5}$

Ensembles, intervalles

Attention, je parle dans cette partie des *ensembles*, et pas des listes en Python.

Les ensembles ça existe aussi en Python, et c'est très similaire aux ensembles en maths :

- Ça se note $\{1, 2, 3\}$ (au lieu de $[1, 2, 3]$)
- Il n'y a aucun doublon dans l'ensemble
- Il n'y a pas de notion d'ordre des éléments

Bien sûr, tout s'adapte aux listes, c'est juste qu'avec des ensembles c'est plus simple pour faire des parallèles avec les maths.

<code>range(a, b)</code>	$\llbracket a, b \rrbracket$
<code>len(A)</code>	$\#A$
<code>{ 2*a for a in A }</code>	$\{2a, a \in A\}$
<code>{ a for a in A if a**2 == a/2 }</code>	$\left\{a \in A, a^2 = \frac{a}{2}\right\}$
<code>{ f(a) for a in A if P(a) }</code>	$\{f(a), a \in A, P(a)\}$ (pas vraiment légal mais on l'a utilisé une fois)

Opérateurs sur les ensembles

<code>A B</code>	$A \cup B$
<code>A & B</code>	$A \cap B$
<code>A ^ B</code>	$A \Delta B$
<code>A - B</code>	$A \setminus B$
<code>A < B <= C > D >= E</code>	$A \subset B \subseteq C \supset D \supseteq E$
<code>b in B</code>	$b \in B$
<code>sum(A)</code>	$\sum A$
<code>prod(A)</code>	$\prod A$

Fonctions

<pre>def f(x): return 2 * x**2 + 5</pre>	Soit $f = x \mapsto 2x^2 + 5$
<pre>def f(x, y): return 2 * x**2 + 5/y</pre>	Soit $f = (x, y) \mapsto 2x^2 + \frac{5}{y}$
<pre>def f(x: int, y: float) -> float: return 2 * x**2 + 5/y</pre>	$f = \begin{cases} Z \times R \rightarrow R \\ (x, y) \mapsto 2x^2 + \frac{5}{y} \end{cases}$

Variables liées et libres

Vous vous rappelez le truc chelou de la marmite, et le fait qu'on n'a pas accès aux variables déclarées dans des fonctions en dehors de celles-ci ?

Et bah y'a tout pareil en maths en fait.

`a = 5`

```
def f(x, y):  
    return a + x*y
```

Ici, en dehors de `f`, impossible d'accéder à `x` ou `y`, ils n'existent pas.

Par contre, on peut accéder à `a` dans `f`, ou en dehors bien sûr.

Et bah en maths aussi c'est pareil :

Soit $a = 5$.

Notons $f = (x, y) \mapsto a + xy$.

Ainsi $x = 666$

Dans la définition de f , on utilise a sans problème, par contre dans la ligne d'après...

WHAT?

Mais *qui* est x ?

Pas de raison que ce soit différent en programmation.

`x` et `y` sont des variables *liées* par :

```
def f(x, y):
```

de la même manière que x et y sont liées par :

$$(x, y) \mapsto$$